Your Data, Your Rules: Extending the Decentralized Web

Jacob Hoffman
web10 Inc.
jacob@web10.app

ABSTRACT

Owning a copy of your posts is not the same as controlling the relationships around them. **web10** brings content, group membership, and service-scoped grants into one user-managed data substrate. Applications write their own queries over a shared ClickHouse store; the node supplies authorized relations before their joins and aggregates execute. A game and a marketplace reuse social-written profiles through this interface without adding a backend or changing the social application. The network design extends the same checks to foreign principals through home-issuer certification. A separate domain-follow prototype preserves a public subscription after its original provider becomes unavailable. Creator-controlled advertising links provide a monetization path without a platform cut of their payouts. A 1,000-group experiment identifies a metadata bottleneck: batching reduces median handler time from 7.64 s to 363 ms, with a cost at one group. The contribution is the combination: user-managed access and application-defined computation on the same shared columnar store.

1 OWNING THE CONTENT IS NOT ENOUGH

What makes your data worth owning is not just the content. It is the graph around it: who follows you, who belongs to your groups, and who you have allowed to read or change what. A creator can download a catalog and still lose the audience that made it valuable. A user can export a profile and still need the next application to rebuild the relationships and permissions around it. Ownership needs to reach the operations performed on the data, not stop at an export file.

The decentralized web has made substantial progress on this problem. Solid separates applications from user-controlled storage and gives users control over access to resources [8, 19]. Mastodon distributes social activity between servers through ActivityPub [13]. AT Protocol separates user repositories from relays and application views, supports provider switching, and lets applications define new record types [3, 12]. Its permission system also supports granular delegation [2]. The remaining design question is not whether applications can reuse data. It is *where they can compute over it, under whose access rules, and with how much new infrastructure.*

Consider a game that needs a leaderboard joined to players' existing social profiles. Fetching reusable records is one part of the job. Maintaining a queryable cross-user view is another. An application can build its own index and database, synchronize records into it, and implement the corresponding read API. web10 supplies a different seam: the application submits the join to the node that already stores the data. The node resolves each service to the relations this reader may access, then executes the application's query. The application needs the data convention and the applicable grants, not a new indexing backend or a change to the application that wrote the profiles.

web10 is a deployed data platform built around that division of labor. Its node exposes generic document reads and writes, identity and access configuration, a constrained query interface, and media delivery. It has no feed endpoint and no platform-owned posts schema. Applications name their own services and write their own queries. Users manage the grants as rows in the same database that holds the content. The platform enforces which relations a query can read; the application developer chooses and tunes the computation over them.

The design brings together five properties:

(1) **Application-independent data.** Posts, profiles, scores, and listings use one document substrate rather than separate application-specific node APIs (§2).
(2) **User-managed access.** Groups, roles, and service-scoped grants express shared access as editable data (§2).
(3) **Authorized composition.** Application-written joins and aggregates operate on reader-filtered inputs in a shared columnar engine (§4).
(4) **Cross-node identity.** A serving node can authorize a foreign principal after home-issuer certification, without creating a second user account (§6).
(5) **An address distinct from an account.** A compatible client can follow a creator-controlled domain and resolve its current public target after a provider change (§6.1).

The first three are exercised by the social application, Snake, and Marketplace. Cross-node access is demonstrated on two isolated nodes with independent signing keys. Domain subscriptions are demonstrated in a separate research client; deployed social follows remain group-based. These are distinct pieces of evidence, reported in §9, not a claim that the complete network design is already a production deployment.

Figure 1 locates the work. In the browser, the social application carries feed construction, group discovery, face and media resolution, request orchestration, and acting-identity state. The SDK transports requests but adds no authority. The node authenticates, checks grants, compiles queries, and prepares media references. ClickHouse holds documents and access metadata; object storage holds media bytes. The client sends SQL and a token, receives rows and authorized media URLs, and fetches the bytes separately. A new composition changes the application's query rather than this arrangement.

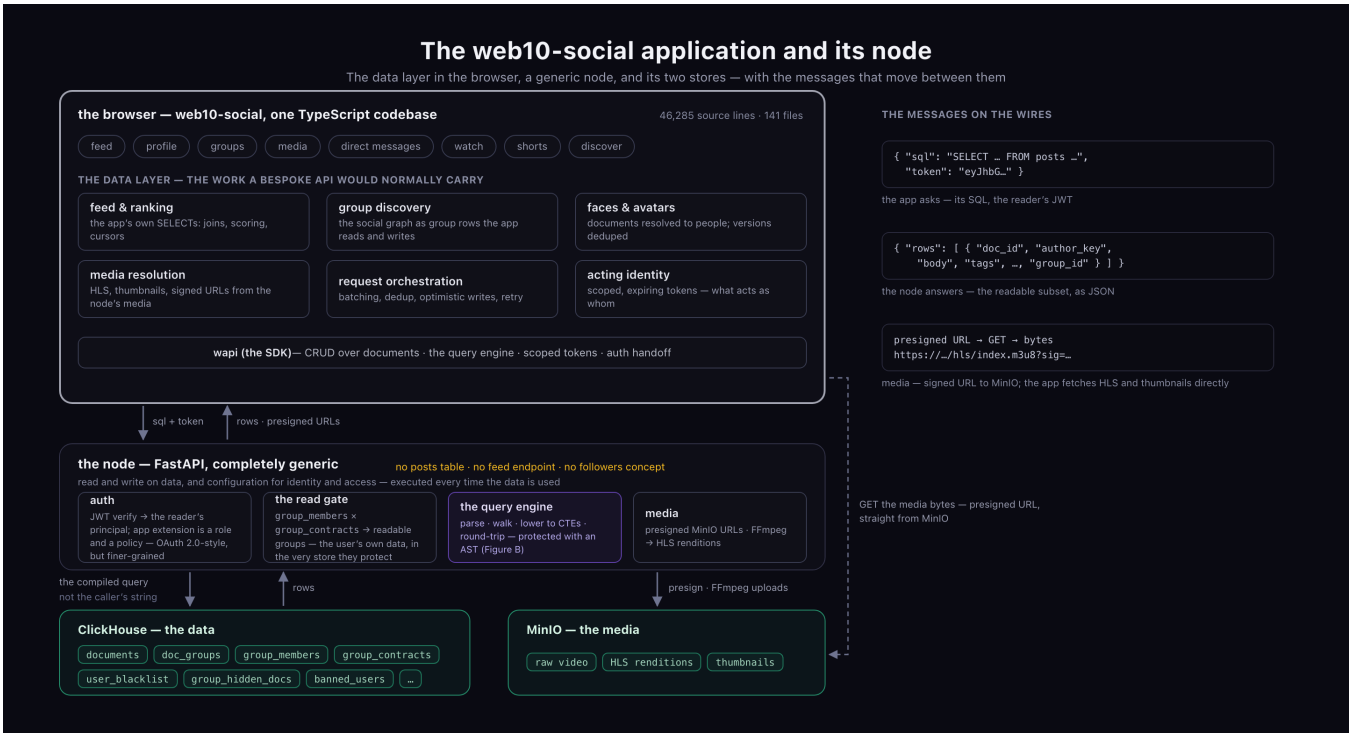


Figure 1: The application, the generic node, and their message paths.

2 THE RULES ARE DATA IN THE STORE THEY PROTECT

A web10 document has an author, an identifier, a service name, and a JSON body. A post and a game score differ in their service string and body convention, not in their storage interface. The node never needs to learn what a score means. Applications agree on the shape of data they share; the node supplies storage and access, not automatic semantic interoperability.

Documents attach to groups through `doc_groups`. A group contract defines roles, and `group_members` assigns a principal to a role. A role is a per-service permission map: for example, `posts: readAll, create, updateOwn` and `comments: readAll`. The reserved principals `anyone` and `authenticated` express access for anonymous and signed-in readers. A publicly readable group has an `anyone` read grant. An invite-only group can therefore remain publicly readable: admission to membership and permission to read are different decisions.

App contracts express permission for an application to use services; group membership and class grants express access to content. Revoking an application's grant does not require leaving a group, and removing a membership does not require revoking an application. The current content gate admits active members for reads and creates; non-member reads and creates use the applicable class grants. Other role-controlled operations check their named permissions. Thus a role map alone is not the complete definition of every content decision.

Users manage these relationships through the same API their applications use. Making a group public writes a grant. Removing access writes a tombstone. Subsequent checks resolve the current metadata rows. This is the sense in which the permission model belongs to the user: the user is the policy administrator for their own hosted data, not merely the recipient of application-specific sharing buttons.

Both content and metadata live in one ClickHouse database per node. The `documents` table is keyed by `(author_key, doc_id)`; the service is stored in `collection_name`. A user's namespace is logical isolation in this shared table, not a physical database per person. Updates insert newer versions, and deletes insert tombstones. Reads rank the complete version history before discarding deleted rows. Discarding tombstones first can expose an older version of a deleted document or grant.

The node operator can read hosted content. Search, ranking, moderation, and audit use plaintext, and default end-to-end encryption is not part of this model. User ownership here means authorship namespaces and control of application and group grants. Operator conduct is governed by terms and applicable law; the API does not cryptographically constrain an operator with physical database access. The design optimizes for user-controlled data policy and useful computation over it, not operator-blind storage.

3 ONE SHARED STORE CHANGES WHERE COMPOSITION HAPPENS

ClickHouse supplies the analytical engine [20]; web10 supplies the access boundary around its use. Housing users together in one node makes a cross-author join a local relational operation. A leaderboard can join scores to profiles, and a discovery view can

rank readable posts across authors, without each application first building its own synchronized copy of those documents.

The choice differs from the pod-oriented interface of Solid. Solid already separates applications from data and supports user-controlled authorization. It does not require content to be end-to-end encrypted, and a pod is not necessarily a separate physical server. The difference relevant here is the exposed unit of composition: resources reached through pod interfaces versus authorized service relations queried together inside one columnar engine.

Figure 2 compares a direct-fetch pod application with web10's node-side composition. In the left-hand design, the application retrieves the resources its feed touches and joins them itself. Wider discovery requires a way to discover and index the relevant resources; caching, query services, and materialized indexes can change those costs. On the right, the data hosted by each web10 node is already in a shared store. The application submits its query to each selected node and can merge the returned results. A query over one node is not a global index over every node.

This is an architectural comparison, not a measured claim that Solid cannot implement discovery or that web10 has demonstrated million-user capacity. The advantage is the local composition service the web10 node supplies out of the box. Physical colocation brings its own costs: read-time version resolution, background merges, point lookups on an analytical engine, and contention between tenants. §8 returns to those costs.

4 YOUR QUERY, OVER THE RELATIONS YOU MAY READ

An application cannot be allowed to query the raw shared table and then have its output filtered afterward. Consider this aggregation:

```
SELECT author_key, count(*)
FROM documents WHERE collection_name = 'comments'
GROUP BY author_key
```

An outer filter cannot undo counts that already incorporated documents the reader was not allowed to see. Authorization has to restrict the inputs to the caller's relational operations.

The application names services: `posts`, `reactions`, `comments`, and `profile`. Its SDK sends SQL and the token to `POST /v3/query`. The node parses the SQL into an abstract syntax tree (AST) and validates the supported read-only statement shape. It walks table references inside the query, subqueries, and caller-defined common table expressions (CTEs). It rejects raw node tables, table functions, and services outside the applicable application permission. The caller does not receive a raw database connection.

For each accepted service, the node constructs a *boundary CTE*: the service's current documents joined to current group attachments, restricted to groups readable by this principal. Blocks, sharing state, hidden-document rules, and node bans apply additional exclusions. The node emits the query from the validated AST and re-parses the result to require exactly one statement. It executes this generated query, not the submitted string. Figure 3 follows the browser's feed query through these steps.

The consequence is compositional. Each service reference already means an authorized input relation when the application's joins and aggregates run. Joining scores to profiles cannot open a private profile merely because the scores are public. Conversely, granting the application use of `profile` does not by itself grant the reader access to every profile group. The connected controls in §9.1 exercise these two gates independently.

This boundary is implemented by the parser, validator, metadata resolution, generated SQL, and the engine's interpretation of that SQL. AST construction is not a proof that every possible SQL shape is safe; parser and engine changes require compatibility and adversarial tests. The architecture places the access decision on the input relations, while correctness depends on faithfully implementing that placement.

PostgreSQL row-security policies [17] provide another placement: the engine applies policies to rows while evaluating queries. web10 constructs reader-specific relations above the engine, using one shared database service account. This is a design choice, not a claim that columnar engines inherently require external enforcement or that row-level security cannot compose. A matched policy-and-workload comparison remains to be measured.

The developer remains responsible for query performance. The node provides authorized inputs, not an automatically efficient ranking algorithm. Resolving those inputs also has a cost of its own. The fan-out experiment in §9.4 shows how metadata round trips can dominate a small document query even when execution in ClickHouse is fast.

5 A NEW APPLICATION WITHOUT A NEW DATA BACKEND

web10-social is the platform's primary application and its most demanding consumer. Text, images, short and long video, audio, groups, and social relationships share the document, group, query, and media interfaces. The browser owns feed definitions, scoring, joins, and cursors. The pinned application snapshot contains 46,285 physical source lines across 141 files under `src/`, excluding tests: 45,846 TypeScript/TSX lines and 439 CSS lines, including comments and blanks. Dependencies and tooling are excluded. The point is the application carried by the generic node, not source size as a measure of generality.

Figure 4 shows a group in that application. Names, member presentation, and post visibility are social UI choices expressed through the generic primitives. Media objects reside outside ClickHouse; the node issues short-lived object URLs and HLS capabilities after the relevant authorization checks. Capability expiry and document access are separate lifecycles: removing a grant does not erase downloaded bytes or instantly invalidate every URL already issued.

Snake and Marketplace exercise reuse rather than another social surface. Snake writes `snake_scores`; Marketplace writes `marketplace_listings`. Both read social-written `profile` documents through the user's grants. Their common dependency is the profile convention, not a new agreement between application backends. The leaderboard query is ordinary composition:

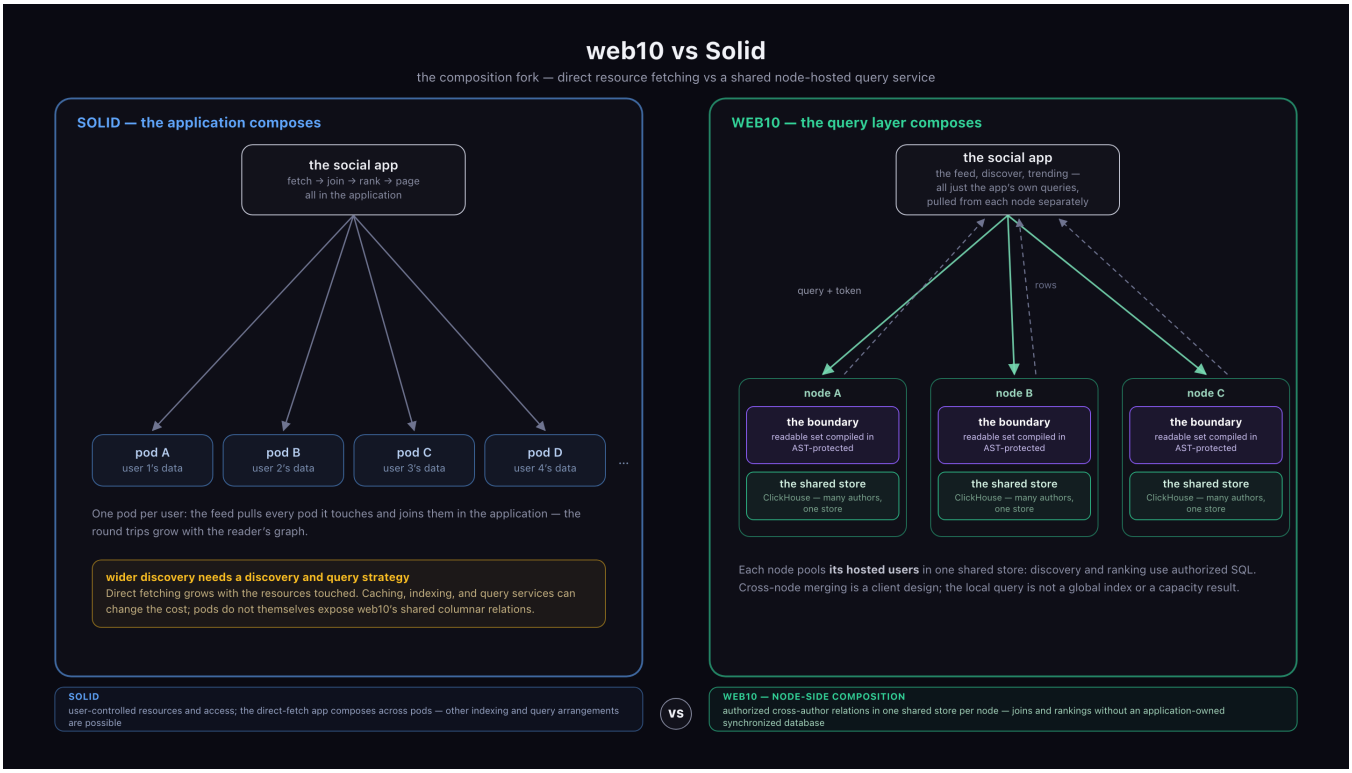


Figure 2: web10 vs Solid: node-side queries and direct pod composition.

```
WITH latest_profiles AS (
  SELECT author_key, body FROM profile
  QUALIFY row_number() OVER (
    PARTITION BY author_key
    ORDER BY updated_at DESC, created_at DESC,
             doc_id DESC) = 1
)
SELECT s.author_key,
       JSONExtractInt(s.body, 'score') AS score,
       p.body AS profile_body
FROM snake_scores s LEFT JOIN latest_profiles p
  ON s.author_key = p.author_key
ORDER BY score DESC LIMIT 50
```

Both service names resolve through the query boundary before the join. An app contract grants use of the required services; group access determines the readable content. OAuth 2.0 supplies the familiar model of a user approving an application's delegated access [9]. web10's concrete vocabulary names services, groups, and operations; OAuth itself does not prescribe how coarse or fine an application's scope vocabulary must be.

Figure 5 locates Mastodon's application-data boundary. Its stock REST API exposes social resources, including profiles through `GET /api/v1/accounts/:id` [14, 15]. A game or marketplace can reuse those profiles. The illustrated integration supplies its own score/listing data layer and combines that data with the profile responses, in application code on the client or server. web10 supplies storage for all three services and executes their authorized joins in the node. This comparison concerns the stock Mastodon interface, not the limits of ActivityPub extensions.

Figure 6 asks a different question: where does a new cross-user query run? The same social application, game, and marketplace appear on both sides. AT Protocol supports custom Lexicons, shared records, custom feeds, and granular permissions. In the illustrated indexed-view design, applications write profiles, scores, and listings to users' repositories. The application can reuse an existing AppView when its available data and API provide the required read. Otherwise, it must design and build its own read backend: choose the ingested records, storage model, queries, and read interface. The figure draws this fork explicitly, with the existing-service branch bypassing custom construction. A custom backend then does three concrete jobs. A synchronization process delivers changes to application webhook code that applies creates, updates, and deletes. A separate application database holds the selected records in queryable tables. Request-serving code queries those tables and returns a page or data to the browser. These responsibilities form an *AppView*-style read backend; they need not be three separate processes or servers.

The path follows the Statusphere tutorial and its pinned implementation [3, 4]. That example uses Tap for record synchronization, SQLite for the application's database, and Next.js server code with SQL query functions to produce the page. Its `account` and `status` tables illustrate an application-designed relational schema. An AppView builder can instead choose a generic JSON table, separate typed tables, or another layout; the engine, keys, indexes, and update strategy are implementation choices, not a storage schema supplied by AT Protocol for that custom view. The figure applies

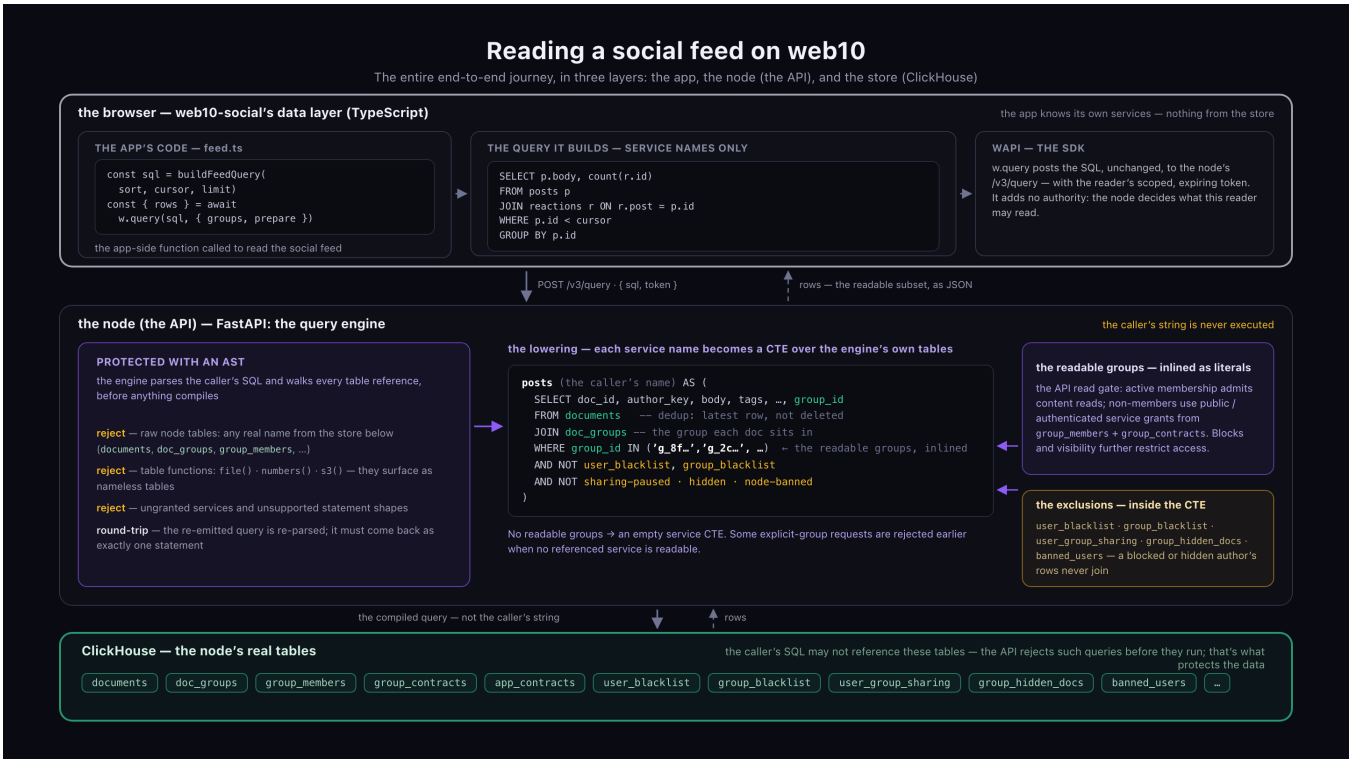


Figure 3: Reading a social feed: application, query compiler, and store.

that architecture to the illustrative score/profile and listing/profile joins; the tutorial itself is a status application. Existing read services, shared indexes, and direct reads are alternatives to building that backend. web10 instead supplies the query API and authoritative document store at the node. Profiles, scores, and listings share the same `documents` table: `collection_name` names the service and body holds its JSON. Group and grant metadata have separate node-managed tables. Applications still design their JSON conventions and queries, but a new service needs no new content table or application-specific read server for this supported composition. The application sends its query and token; the node constructs authorized input relations, executes the join, and returns rows. For this supported composition, the developer maintains no separate synchronization process, webhook handler or copied-table database. The advantage is supplied infrastructure for authorized computation, not exclusive extensibility or a measured performance difference.

The screenshots in Figure 7 show the resulting reuse: one social-written identity appears beside a genuinely earned score and a marketplace listing. These are direct browser captures with synthetic accounts. The corresponding consent, revocation, and exact-result checks are reported in §9.1.

6 ACROSS NODES: IDENTITY IS VERIFIED, ACCESS STAYS LOCAL

The network design gives a principal a provider-qualified identity: `provider/username`. A user has a home account, but a group on another node can grant that principal access without creating a local namesake account. The serving node must establish two things separately: whether the home issuer recognizes the token, and whether this principal has access under the serving node's own rules.

The `/certify` exchange answers the first question. The serving node asks the token's home issuer over HTTPS and caches a successful answer for that exact JWT until its expiry. The answer verifies identity; it is not a grant. Each subsequent content read still resolves current access at the serving node. Email's sender checks offer a useful analogy: SPF and DKIM establish facts about an arriving message's origin, not a right to bypass the recipient's rules. The mechanisms differ, but the separation between verification and authorization is the same.

Figure 8 illustrates the intended client-mediated read model. The client groups its targets by provider, queries the selected nodes with its home token, and merges independently authorized results. One node never needs database credentials for another node's store. ClickHouse's `remote()` and `cluster()` are therefore not the cross-operator interface. A remote API response is content to render or merge, not SQL to execute locally and not evidence of a local grant.

The two-node experiment demonstrates certification and owner-managed foreign grants on both document and query reads. Complete client-side feed merging, foreign enrollment, and independent production-node provisioning remain integration work. Local paths still use legacy bare principal keys while the

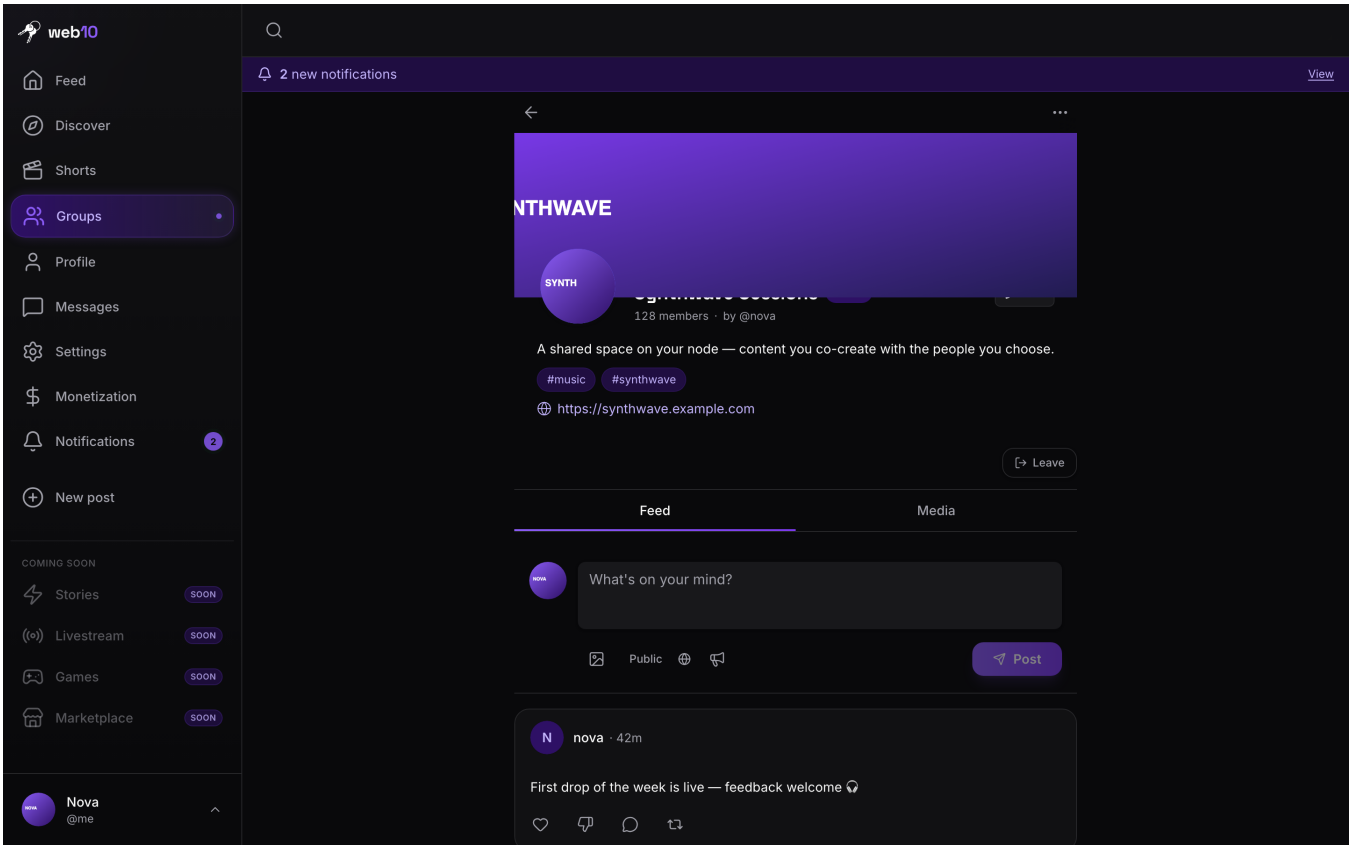


Figure 4: A social group built on generic documents and grants.

exercised foreign paths use qualified keys; the coordinated identity migration is not complete. The figure describes the network arrangement, not a measured production cross-node feed.

6.1 A public subscription that outlives an account

An account principal and a public address serve different purposes. The domain prototype uses a creator-controlled domain as the public address. A TXT record at `web10.domain` names its target: `web10.v=1; provider=node; user=username`. The creator authenticates to the target node and calls `POST /v3/domains/verify`. The node performs the DNS lookup itself and checks that the record names this node and this account before storing a verified alias. DNS discovers the account; it does not replace the account's authentication or group principal.

The reader stores the domain in a private document at the reader's home. On a later read, the prototype resolves that domain, checks the target's verified alias, and requests public content. When the creator changes provider, the creator changes the TXT pointer and verifies the new account. The saved subscription does not change. The old node cannot edit a DNS zone the creator controls, and the move need not ask the old node to release its stored alias.

Figure 9 follows this indirection through a provider change. The fan's subscription remains in the fan's data; the creator changes the address's target. Neither old content nor private grants are transferred by DNS. The demonstrated benefit is continuity of *public discovery*, not continuity of private authority. The research client implements this behavior; production social follows are unchanged. It assumes retained domain control and available DNS, reader home, and replacement provider. Domain expiry or theft can redirect the subscription.

7 ADVERTISING WITHOUT OWNING THE CREATOR'S PAYOUT

A creator platform also needs an economic model. web10's current approach is an advertising rail, not a payment processor. A creator's offer is data: the creative, offer kind, destination link, call to action, and disclosure. Affiliate and sponsor payments settle outside the node. web10 does not rewrite the creator's link or take a percentage of those payouts. This is a policy of this rail, not a claim that creators cannot monetize on other decentralized systems.

Figure 10 separates the creator's pinned offer from the node operator's overlay. Result preparation can attach both to one document; the node's overlay does not replace the creator's offer. The operator configures overlay density, with a default of 10%. Selection is deterministic per reader and inventory rotation distributes

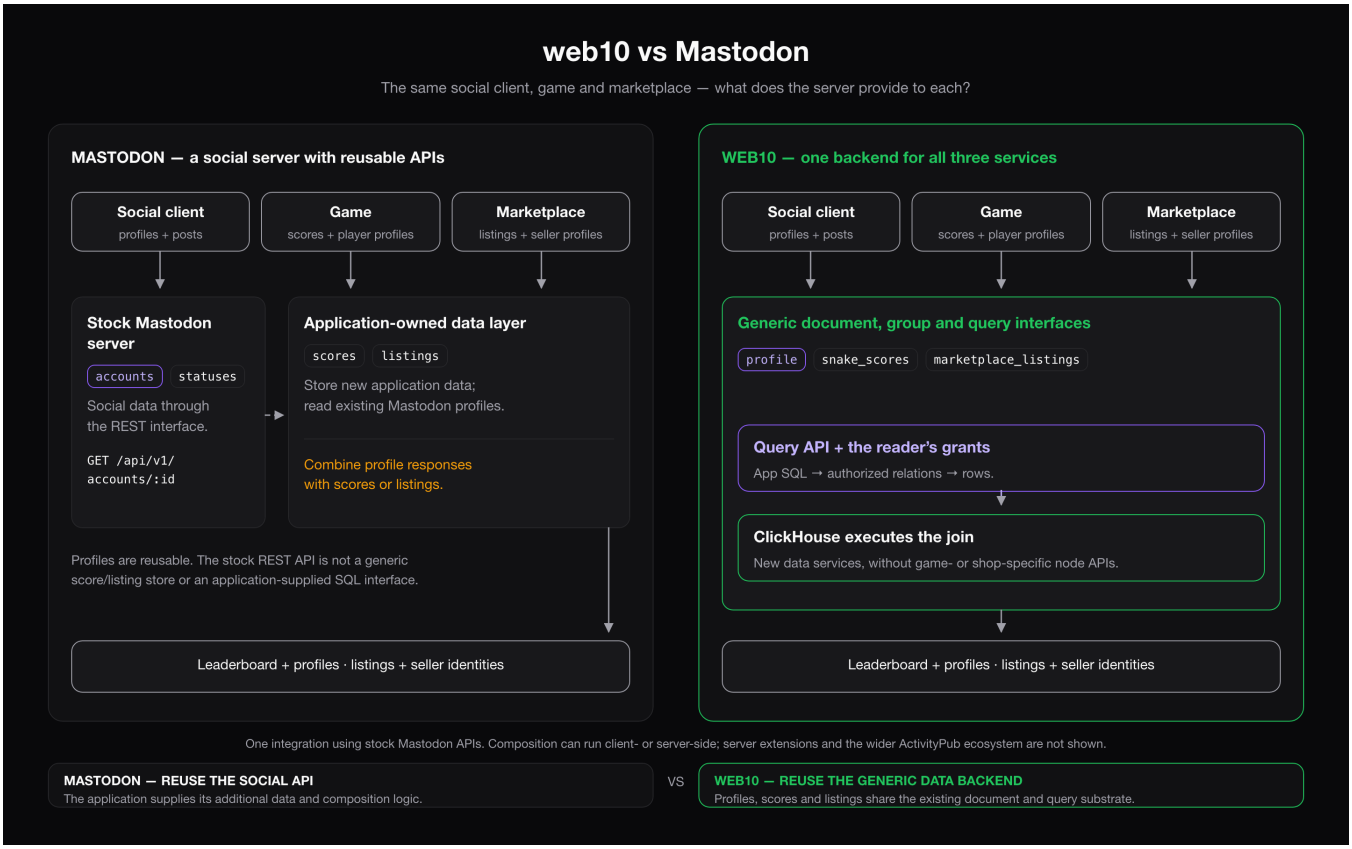


Figure 5: web10 vs Mastodon: the application-data boundary.

the selected node ads. Ad documents remain subject to their own access checks; an authorized post is not blanket authority to read any creative.

The figure's dense creator layer (80–90%) and sparse node layer (5–10%) describe proposed operating choices, not measured advertising exposure. Creators choose where to pin their offers, and operators configure their overlay. The mechanism is service-agnostic: another application can use eligible documents and render the returned ad fields. web10-social currently builds the catalog; the interoperability experiment does not demonstrate ad rendering in Snake or Marketplace.

The node operator can sell its own overlay inventory, as a publication sells advertising space. Creator affiliate payouts, operator inventory revenue, and hosting or platform fees are separate revenue lines. There is no measured operator revenue share or marketplace outcome in this paper. An ad exchange, payment processing, and creator-defined ad layouts are future layers rather than requirements of the current rail.

Figure 11 illustrates the incentive the network design is intended to create. Six hypothetical creators choose among six hypothetical nodes with different prices, moderation policies, and overlay densities. Some switch when their preferences change; others respond when a node changes its terms. Self-hosting is an outside option, with infrastructure and operating costs even when the software is free. If a compatible domain subscription survives a provider change, an operator has less leverage from controlling the old account address. This makes terms a plausible axis of competition. Whether such a market forms, and how much switching cost remains, are economic questions not answered by the correctness experiments.

8 LESSONS FROM BUILDING AND OPERATING THE NODE

8.1 Generality has to survive the primary application

The social application repeatedly created pressure for bespoke node features. Four were built and then removed: a dedicated feed endpoint, a group-identity table and endpoint, a followers-group check in the session oracle, and an ad path hard-coded to a collection name. Their replacements were, respectively, an application query, documents in an app-named service, application-side session logic, and generic tags and group attachments.

The test was practical: would a notes application, music application, or shop use this node operation? If not, the concept belonged in the application. Holding this line moved more orchestration into the browser. It also made the later game and marketplace possible without extending the node for either of them. The generic core

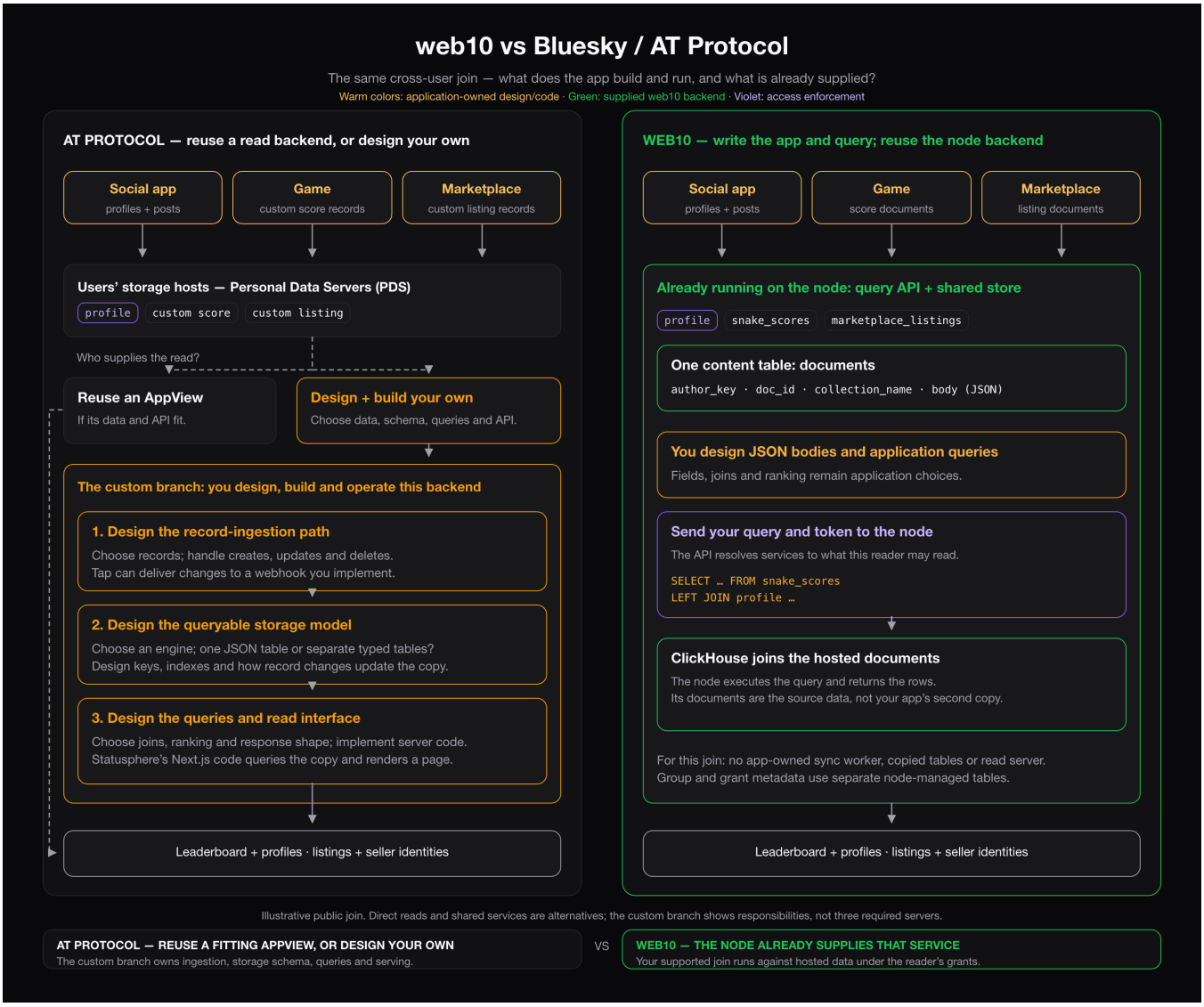


Figure 6: web10 vs Bluesky / AT Protocol: the read backend, unpacked.

emerged from removing plausible shortcuts, not from keeping the first design untouched.

8.2 A schema repair is not ready until readers can use it

The connected interoperability work encountered a fresh-stack failure: `group_contracts.tags` was absent when an early request needed it. The runner had launched the API before initializing the database. Starting storage, initializing schema, and then starting the API let the existing boot migration operate against real tables. This was a runner ordering repair, not closure of the project-wide fresh-start issue.

The general requirement is stronger than eventually adding a column: required schema must exist before readers are admitted. A fresh database and its first request need a separate test from the steady-state system. Deferring an upgrade is safe only when no admitted request depends on it.

8.3 A shared analytical engine still pays operational costs

`ReplacingMergeTree` removes obsolete versions during background merges, but reads cannot wait for those merges to establish correctness. They resolve versions explicitly. Tombstones participate in that ranking; they cannot be filtered away first. Documents attached to multiple readable groups also create join multiplicity, so applications must choose the deduplication semantics of counts and pagination.

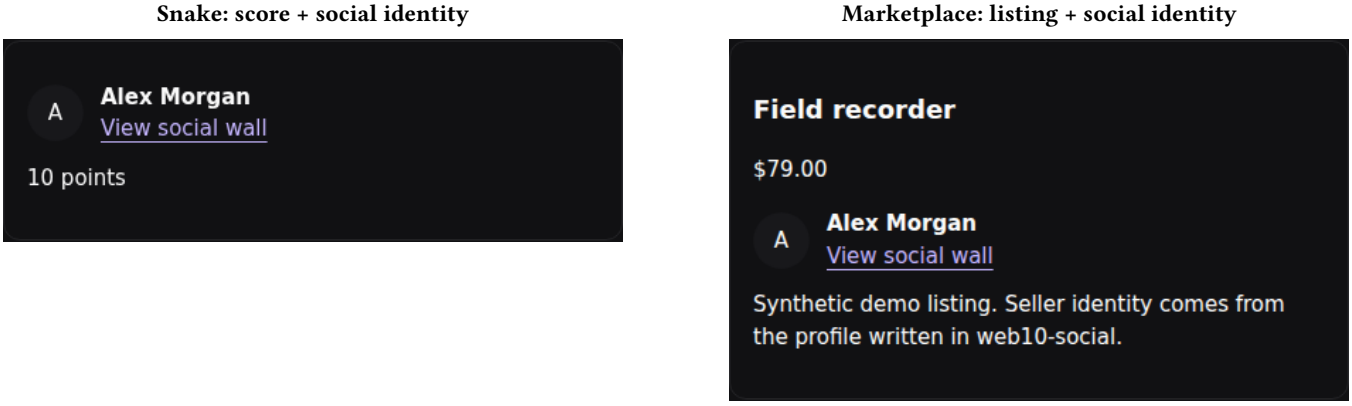


Figure 7: One social-written profile reused by two applications.

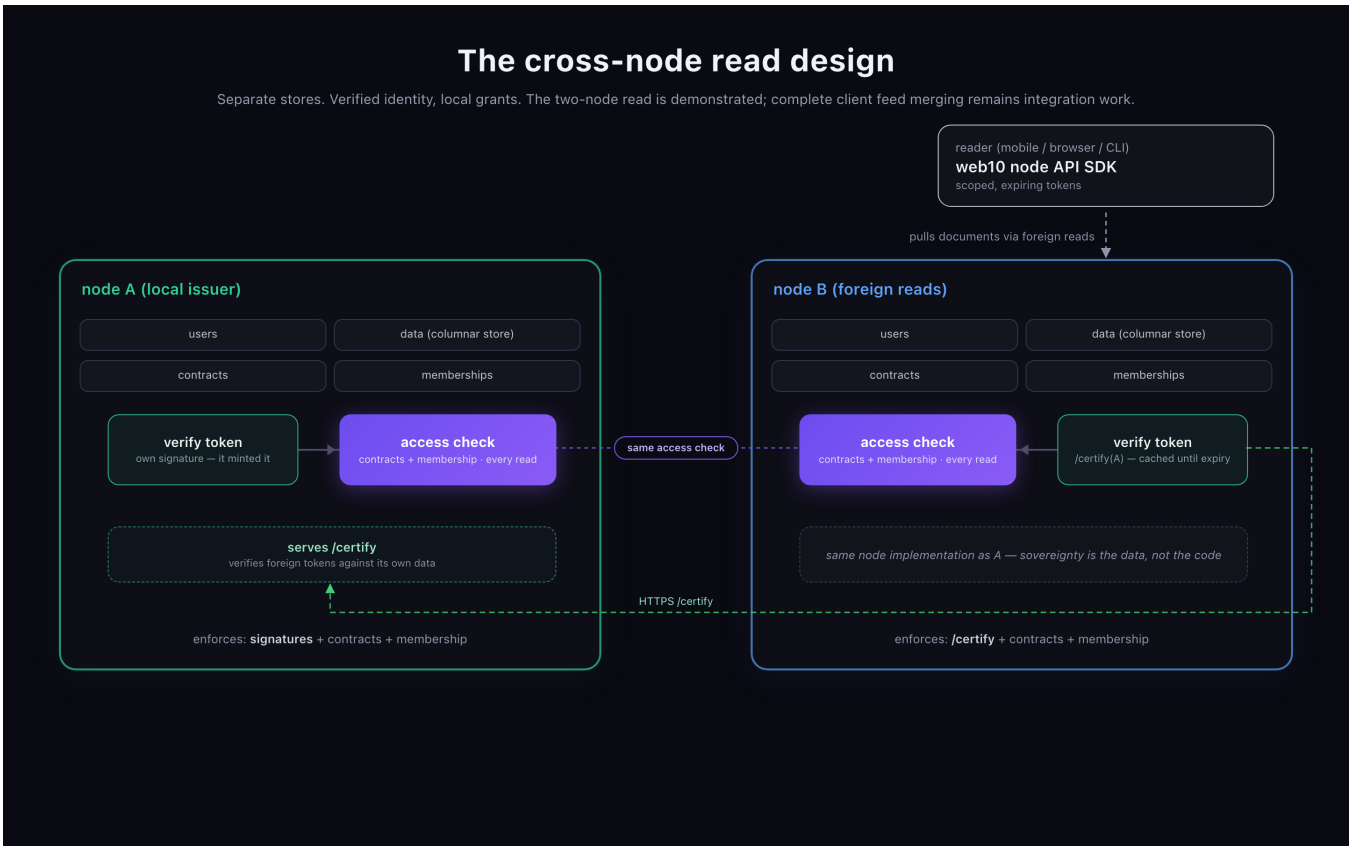


Figure 8: Cross-node read design: certify identity, then apply local grants.

The same shared engine handles metadata lookups and analytical scans. Merge work consumes resources, small point queries pay fixed round-trip costs, and tenants share the node's capacity. Query freedom therefore creates an operational obligation to manage resource use. The first fan-out measurement found a bottleneck in repeated permission-metadata queries rather than document scanning. Reducing those round trips was more valuable for that workload than changing its document query.

9 EVIDENCE

The evaluation answers four mechanism questions and reports one deployed operating point. The mechanism experiments use synthetic data on isolated stacks. The operating point is a read-only production observation. None of these is a concurrent-capacity trial or a performance comparison against another platform.

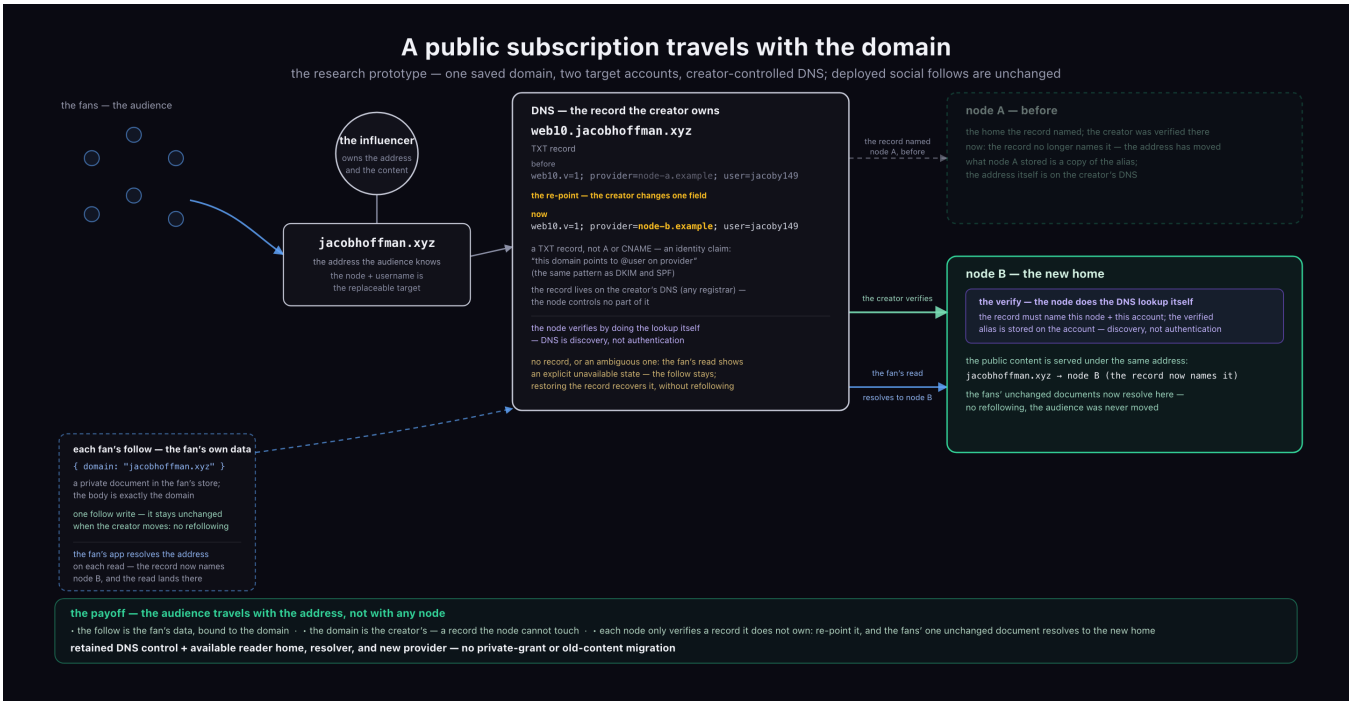


Figure 9: The domain-follow prototype: a new target, the same subscription.

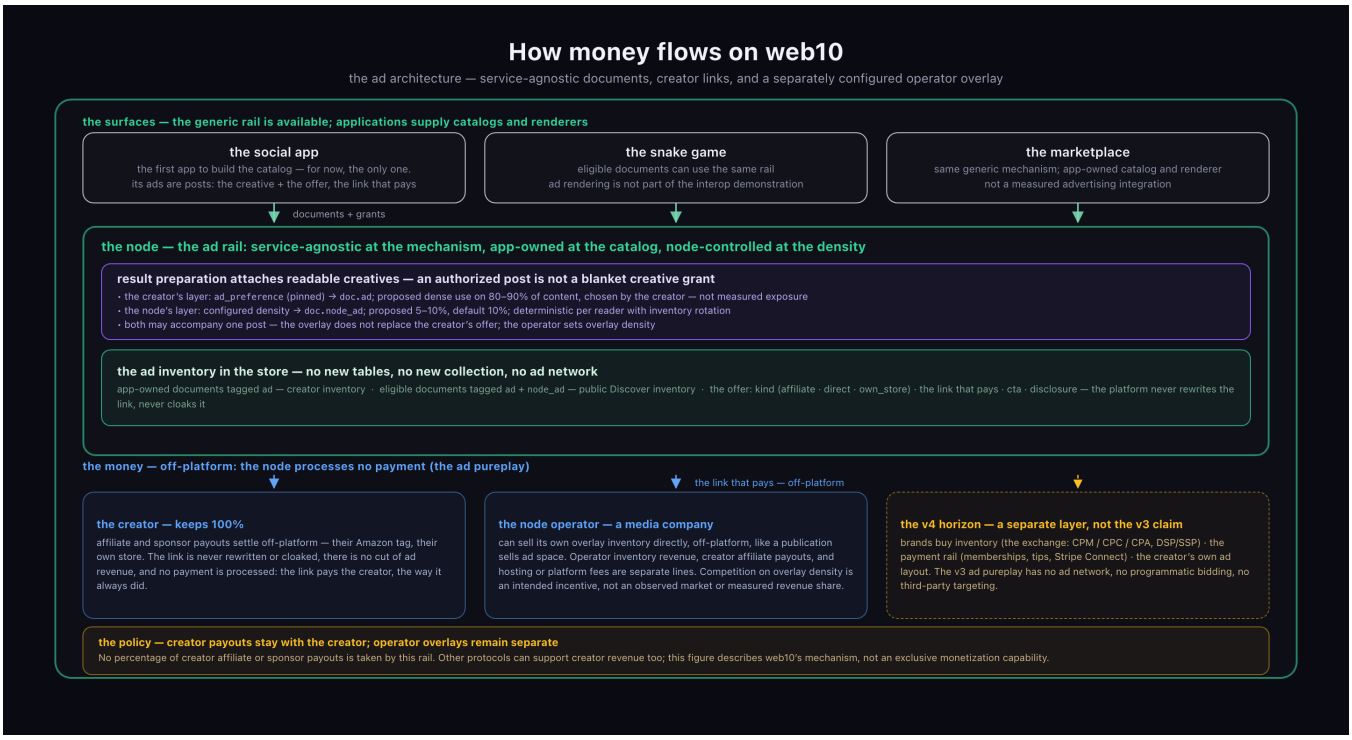


Figure 10: The ad rail: creator offers, operator overlays, off-platform payouts.

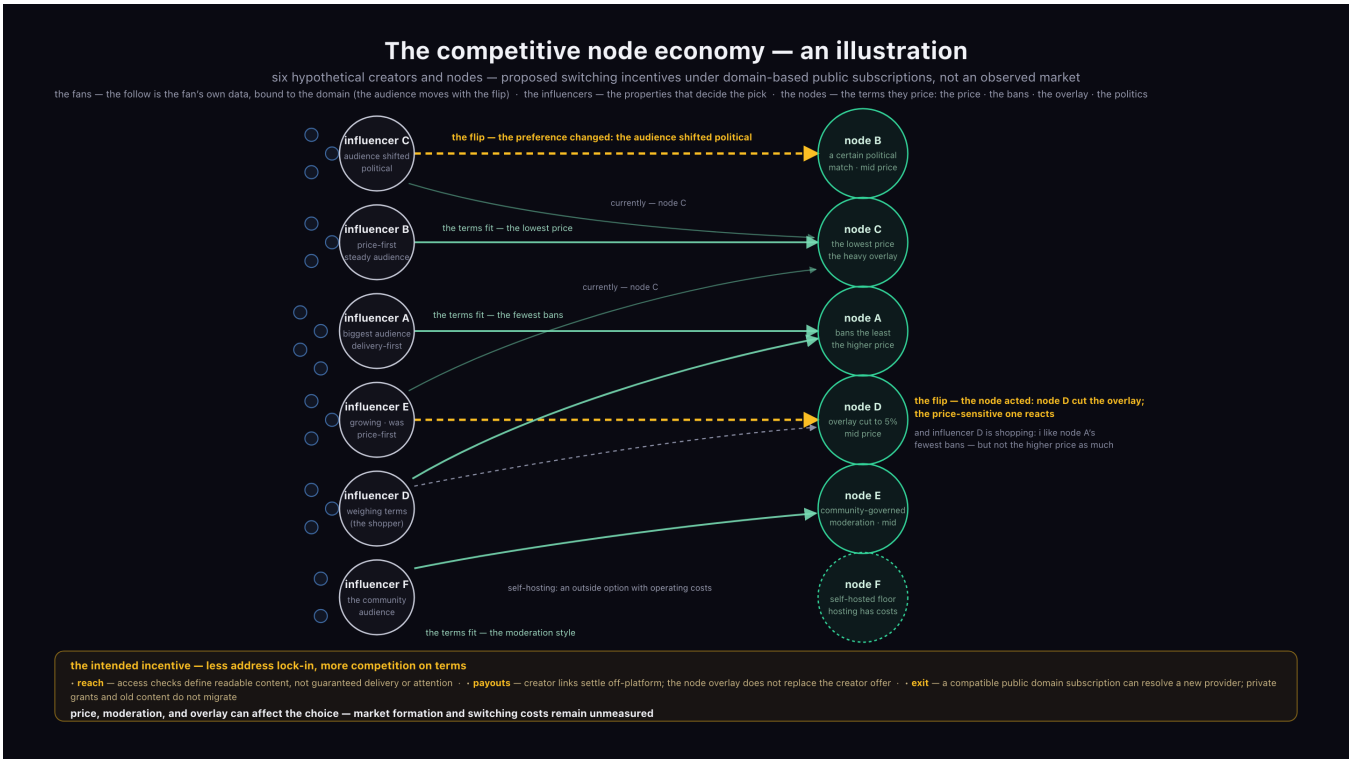


Figure 11: An illustrative node market with changing terms and preferences.

9.1 Can another application reuse social-written data?

In the browser scenario, a fresh user logged into web10-social through its authenticator popup and changed the profile name to Alex Morgan. The user approved Snake's service request, earned and saved ten points, and saw the joined social identity on the leaderboard. Revoking the app contract made a new query return 403; the leaderboard cleared the result and displayed an access error. After sign-out, Marketplace requested its permission through the popup, published a listing, and showed the same social-written identity. Reloading the detail URL retained the correct listing. API assertions compared profile bodies, authors, counts, and document identifiers, so a username fallback could not pass.

A separate three-account HTTP case checked exact two-author score/profile and listing/profile joins. Missing application permission for `profile` denied the queries; granting it enabled them; revocation denied them again. A separately labeled private-profile fixture tested the group gate: membership enabled the inner join, and removal emptied that join while scores remained readable.

The final fresh-stack run on 2 November 2026 passed desktop (1440 pixels), mobile (375 pixels), and HTTP correctness: three cases, zero retries, zero skips. The host was a four-vCPU, 8-GB DigitalOcean sfo2 instance running ClickHouse 24.8.14.39; browser tooling used Playwright 1.63.0 and Chrome for Testing 153. The artifact retains the exact base-plus-patch source, overlay, configurations, image identities, outcomes, and 16 captures.

Both demo applications share a browser origin; the HTTP control supplies a distinct application Origin. These results establish the named consent, reuse, and revocation behaviors, not distinct-origin browser isolation between the demos or independent-developer adoption. The applications know the profile convention. Media delivery, full social feed timing, and a revocation-latency bound were not measured.

9.2 Can a foreign principal read, lose access, and read again?

Two API processes used separate ClickHouse and MinIO instances and independently generated HS256 signing secrets. The serving node reached the home issuer through real HTTPS with an ephemeral test CA and issuer hostname verification. Bob created a private group and document on B. Alice's A-issued token was denied on B's document and query paths. Granting membership to B's distinct, same-named Alice still denied A's Alice. Granting the exact foreign principal enabled both reads of the exact document; removal denied both; regrant restored both.

All lifecycle reads reused the same JWT. The issuer recorded one successful certification across that lifecycle, while B reapplied access checks on every request. A forged foreign-provider token signed with B's key was rejected; repeating it used a negative cache entry rather than another issuer call. A separate Origin-bearing query control returned 403, 200, and 403 around a qualified foreign app-contract grant and revocation. Its app-contract setup was a service-function fixture, not browser consent.

The unchanged scenario passed on clean merged source with an empty patch. The artifact retains the source identity, 34 status events, configuration, image identities, issuer traffic, and verified cleanup. Docker-internal addressing and a test CA are fixtures. Production key provisioning, complete principal migration, foreign self-join, multiworker/restart behavior, and public deployment remain outside this result. Cached certification also does not establish immediate issuer-side revocation.

9.3 Can a public subscription survive the old provider's loss?

The separate domain client stored the creator's normalized domain in a private `domain-demo-follows` document at the reader's home. On desktop (1440×1000) and mobile (375×812), the creator published a TXT record and verified the old account through actual local UDP DNS. The reader clicked Follow once and read its public post. The harness then stopped the old API and TLS frontend.

The creator pointed DNS to a new account and verified it without contacting the old provider. Reader reload and ordinary login displayed the new account's public content with the identical saved follow ID and body and exactly one follow creation. No old content was migrated. The independent private controls returned 200 for the explicitly granted old private source and 403 for the new target's private content. The public pointer did not transfer private authority.

Missing or ambiguous DNS cleared displayed content without removing the subscription. Restoring DNS recovered it without refollowing. Delayed real responses at five identity-transition boundaries verified that logout and account changes prevented stale UI updates. DNS, alias, and public-content requests carried no credentials. The artifact retains the clean committed integration, exact inputs, ten captures, request outcomes, and cleanup.

The prototype permits two configured lab origins. It uses TTL-zero local DNS, a loopback HTTP DNS bridge, explicit app/private-grant fixtures, and browser acceptance of ephemeral certificate errors. Public DNS propagation, arbitrary-provider onboarding, reader-home failure, production social integration, and domain theft or expiry are not demonstrated. The result is public-subscription continuity under the stated availability and domain-control assumptions.

9.4 Where does readable-group resolution become expensive?

The query path resolves readable groups before executing the application's relational operations. The experiment varies candidate-group count K to compare point metadata lookups with batched resolution. Figure 12 plots readable-group time, not browser latency or complete feed assembly.

Setup. One sequential trial ran in DigitalOcean sfo2. The `s-4vcpu-8gb` host provided four vCPUs and 8 GB nominal RAM. It ran Ubuntu 24.04, Linux 6.8, and Docker 29.1.3, with one Uvicorn process and ClickHouse 24.3.18.7. Seed 42 created 1,000 creators with one small post each and a reader who joined all their groups. Those groups also permitted public reads. Personal queries supplied $K = 10$, 100, and 1,000 on this same dataset. A separate discover query used one board group and ten additional posts. Twenty samples in each of 20 cells produced 400 measurements: 320 HTTP requests and 80 direct engine executions. All HTTP requests returned 200.

Variants. The per-group resolver issues point metadata queries. The batched resolver fetches memberships, principal-class grants, and role definitions in three queries, then resolves access in memory. Variant B executes the real document query. Variant C substitutes a fixed 20-row scan result while retaining policy resolution and compilation. Variant A precompiles the known-readable projection and times execution directly in ClickHouse outside the HTTP handler.

The document query selects IDs and bodies, orders by creation time and ID, and limits to 20 rows. It omits engagement joins, media preparation, and browser assembly. HTTP requests omit Origin and do not exercise the browser-origin app-contract gate. C is not an output-equivalent execution, and A excludes authentication, compilation, and serialization.

Finding. From $K = 10$ to $K = 1{,}000$, per-group median readable time rose from 75.03 ms to 7,535.71 ms (100.44×); the corresponding p95 values were 93.30 ms and 8,504.61 ms. Batched readable time rose from 23.47 ms to 213.98 ms (9.12×), with p95 values of 31.17 ms and 284.03 ms. At $K = 100$, the respective readable medians were 764.39 ms and 41.78 ms. At $K = 1{,}000$, batching reduced median handler time from 7,644.66 ms to 363.31 ms (21.04×), and readable time by 35.22×.

Stubbing document execution preserved the separation: at $K = 1{,}000$, C's readable medians were 8,388.39 ms per-group and 222.57 ms batched. A's direct execution medians ranged from 28.6 ms to 54.2 ms across the personal queries. Repeated metadata work, rather than the simplified document scan, dominated the unbatched path. The result supports using the batched resolver for large candidate sets; it does not imply the measured endpoint had already switched to that resolver.

Tradeoff. On the one-group discover query, per-group resolution was faster: 9.53 ms readable versus 25.37 ms batched. The three fixed round trips cost more than a small number of point checks. Batching also did not make cost constant: batched median handler time grew 4.47× across the 100-fold increase in K.

The trial had no concurrency sweep, randomized cell order, or explicit warm-up exclusion. Twenty samples yield preliminary within-run percentiles, not independent repeated-trial characterization. The artifact retains the setup, source, report, and per-cell summary receipts; it does not retain all raw per-sample measurements. Success statuses alone do not establish permission equivalence across policies. $B - A$ is a pipeline-versus-scan comparison, not a causal authorization-overhead estimate. An earlier 100-creator trial used a different host and snapshot and is not pooled with this one.

9.5 What does the deployed operating point show?

A read-only production observation on 28 October 2026 recorded 597 account rows, 593 live documents across 30 distinct authors and ten services, 49 group contracts, and 1,298 membership rows.

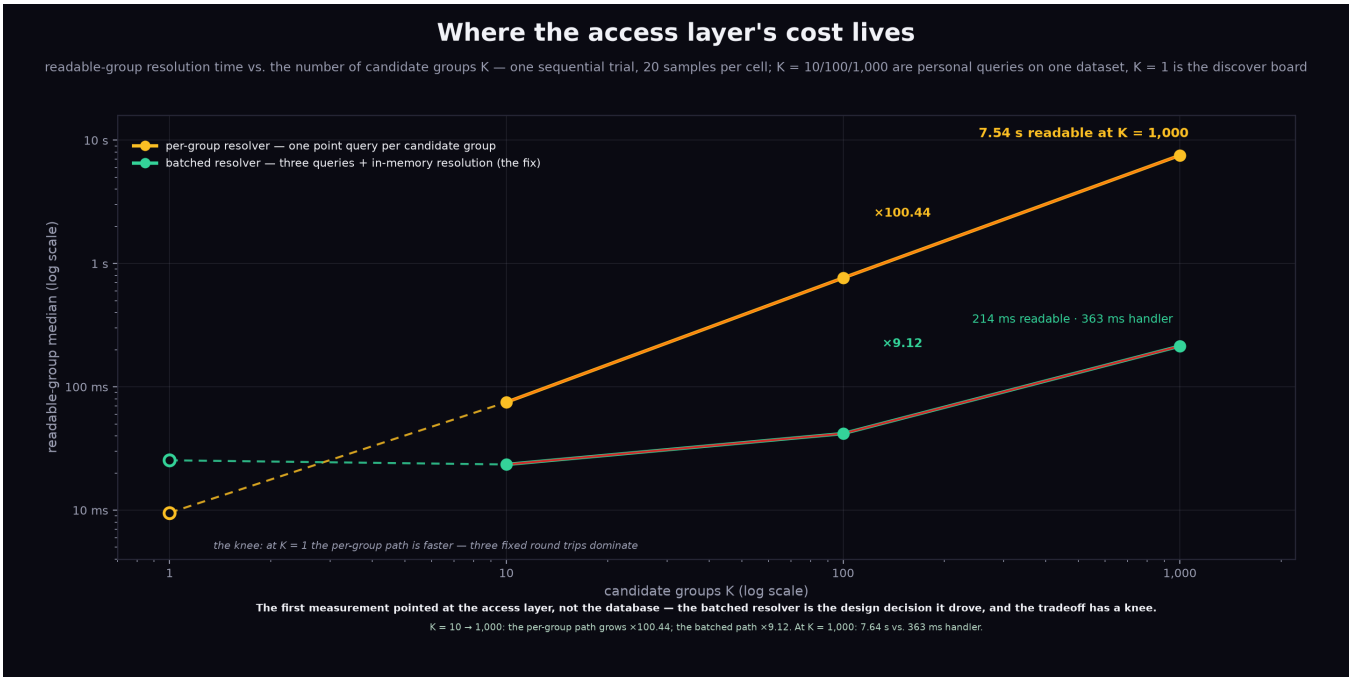


Figure 12: Readable-group resolution cost and its small-K tradeoff.

Nine authenticated browser navigations across following feed, music, and groups returned HTTP 200, retained the session, and produced no uncaught page errors. The following feed rendered 20 posts in a median 5,396 ms of full-page readiness.

This dated operating point establishes deployment and observed use, not a large active-user population or capacity bound. The host also ran a development stack; the snapshot has no activity window. Full-page readiness includes frontend startup and network work and is not the query-handler time measured above. The 5.4-second result makes the next measurement concrete: dissect complete page assembly, including follow-up engagement reads and media delivery, rather than extrapolating from the simplified query.

10 POSITIONING THE COMBINATION

User-controlled storage and interoperable records. Solid [10, 18, 19] is the closest antecedent for separating applications from user-controlled storage. WAC supports resource access and group agents; ACP includes agent, client, and issuer matching [5, 7]. web10 does not introduce user-managed authorization into an otherwise permissionless field. It combines service/group grants with a shared, node-hosted SQL interface. The comparison in Figure 2 concerns that composition placement, not a universal limit on pod query implementations.

ActivityPub defines federated activity exchange [13]. AT Protocol separates repositories, relays, and AppViews and supports extensible Lexicons [12]. Its current permissions specification scopes public-repository writes by collection and action and supports remote-service RPC permissions [2]. These are substantial forms of extensibility and delegation. The Statusphere example also shows how application-owned synchronization and database queries construct a new cross-user view [3]. web10 supplies the shared query service at the storage node and includes group-controlled private document access in that same substrate. This can reduce infrastructure for an application needing those operations; it also places query execution and resource contention at the node.

Authorization and shared databases. User-specific grants and reader-filtered relations are established ideas. Multi-tenant systems have long shared tables under logical tenant identifiers [1, 11, 21]. PostgreSQL policies place row access inside query execution [17]. Zanzibar addresses relationship-based authorization and consistency across services [16], while TAO provides an operational precedent for social-graph storage and read-time privacy checks [6]. web10 joins these concerns in a different deployment unit: end-user data and grant metadata in one columnar database, with application queries compiled against authorized service relations. It does not claim Zanzibar's consistency guarantees or a measured advantage over an equivalent engine-level policy.

The contribution. The system contribution is this integration, demonstrated through cross-application reuse and isolated cross-node authorization, with a measured metadata fan-out tradeoff. The advertising rail is an application of the substrate, not evidence that other decentralized protocols prohibit creator revenue. The market diagram describes a proposed incentive structure, not an economic result.

11 CONCLUSION

User control becomes more useful when applications can compute over the data without rebuilding its storage and authorization machinery. web10 makes that computation a node service. Users manage documents, groups, and grants in a shared store; applications supply queries; the node supplies reader-filtered relations before joins and aggregates. The game and marketplace demonstrate the consequence: social-written profiles become inputs to other applications without a new node endpoint or a separate application data backend.

The network experiments extend different parts of that result. A foreign principal gains and loses access under a single cached certification, because the serving node still checks its own current grants. A separate domain-follow client retains its public subscription after the old provider goes dark, without transferring private authority. These mechanisms address account boundaries and public address continuity; they do not yet establish a complete production cross-node product or account migration.

The cost experiment identifies a concrete design constraint. At 1,000 candidate groups, batching reduced median handler time from 7.64 s to 363 ms, but cost more at one group. Shared columnar execution does not remove the cost of deciding which inputs a reader may use. The next evaluation needs independent repeated trials, full-feed and concurrent workloads, and a matched shared-table policy baseline with equivalent access semantics. Production cross-node integration, revocation-latency bounds, and broader parser/media correctness remain separate gates.

The central result is a working division of labor: the user controls the access relationships, the application controls the computation, and the node provides the common place where the two meet.

REFERENCES

[1] Stefan Aulbach, Torsten Grust, Dean Jacobs, Alfons Kemper, and Jan Rittinger. 2008. Multi-Tenant Databases for Software as a Service: Schema-Mapping Techniques. In *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data (SIGMOD '08)*. ACM, 1195–1206.

[2] Bluesky Social PBC. 2026. AT Protocol: Permissions. Protocol documentation. https://atproto.com/specs/permission Accessed November 6, 2026. Resource Types and Permission Sets.

[3] Bluesky Social PBC. 2026. Create a Social App. AT Protocol tutorial. https://atproto.com/guides/statusphere-tutorial Accessed November 6, 2026. Parts 1, 2, 4, and 5.

[4] Bluesky Social PBC. 2026. Statusphere Example Application. Source code. https://github.com/bluesky-social/statusphere-example-app/tree/dbb97738caa965c4b04efd281bfa1d2fbe4fffb3 Revision dbb97738caa9; accessed November 6, 2026.

[5] Matthieu Bosquet. 2022. Access Control Policy (ACP). W3C Solid Community Group, Editor's Draft. https://solid.github.io/authorization-panel/acp-specification/

[6] Nathan Bronson, Zach Amsden, George Cabrera, Prasad Chakka, Peter Dimov, Hui Ding, Jack Ferris, Anthony Giardullo, Sachin Kulkarni, Harry C. Li, Mark Marchukov, Dmitri Petrov, Lovro Puzar, Yee Jiun Song, and Venkateshwaran Venkataramani. 2013. TAO: Facebook's Distributed Data Store for the Social Graph. In *Proceedings of the 2013 USENIX Annual Technical Conference (USENIX ATC '13)*. USENIX Association, 49–60.

[7] Sarven Capadisli. 2022. Web Access Control. W3C Solid Community Group, Version 1.0.0-cr-1. https://solidproject.org/TR/wac

[8] Sarven Capadisli, Tim Berners-Lee, and Kjetil Kjernsmo. 2024. Solid Protocol. W3C Solid Community Group, Draft Community Group Report, Version 0.11.0. https://solidproject.org/TR/protocol

[9] Dick Hardt. 2012. The OAuth 2.0 Authorization Framework. IETF RFC 6749, Standards Track. https://doi.org/10.17487/RFC6749

[10] Joachim Van Herwegen and Ruben Verborgh. 2024. The Community Solid Server: Supporting Research and Development in an Evolving Ecosystem. *Semantic Web* 15, 6 (2024), 2597–2611. https://doi.org/10.3233/SW-243726

[11] Dean Jacobs and Stefan Aulbach. 2007. Ruminations on Multi-Tenant Databases. In *Proceedings of BTW 2007*. 514–521.

[12] Martin Kleppmann, Paul Frazee, Jake Gold, Jay Graber, Daniel Holmgren, Devin Ivy, Jeromy Johnson, Bryan Newbold, and Jaz Volpert. 2024. Bluesky and the AT Protocol: Usable Decentralized Social Media. In *Proceedings of the ACM CoNEXT-2024 Workshop on the Decentralization of the Internet (DIN '24)*. ACM, 9. https://doi.org/10.1145/3694809.3700740 Extended version: arXiv:2402.03239.

[13] Christine Lemmer-Webber, Jessica Tallon, Erin Shepherd, Amy Guy, and Evan Prodromou. 2018. ActivityPub. W3C Recommendation. https://www.w3.org/TR/activitypub/ Editors: Christine Lemmer-Webber and Jessica Tallon.

[14] Mastodon contributors. 2025. Getting Started with the API. Mastodon documentation. https://docs.joinmastodon.org/client/intro/ Accessed November 6, 2026.

[15] Mastodon contributors. n.d.. Accounts API Methods. Mastodon documentation. https://docs.joinmastodon.org/methods/accounts/ Accessed November 6, 2026. Get account.

[16] Ruoming Pang, Ramon Caceres, Mike Burrows, Zhifeng Chen, Pratik Dave, Nathan Germer, Alexander Golynski, Kevin Graney, Nina Kang, Lea Kissner, Jeffrey L. Korn, Abhishek Parmar, Christina D. Richards, and Mengzhi Wang. 2019. Zanzibar: Google's Consistent, Global Authorization System. In *Proceedings of the 2019 USENIX Annual Technical Conference (USENIX ATC '19)*. USENIX Association. https://www.usenix.org/system/files/atc19-pang.pdf

[17] PostgreSQL Global Development Group. n.d. PostgreSQL Documentation: Row Security Policies. PostgreSQL documentation. https://www.postgresql.org/docs/current/ddl-rowsecurity.html Accessed October 2026.

[18] Andrei Sambra, Amy Guy, Sarven Capadisli, and Nicola Greco. 2016. Building Decentralized Applications for the Social Web. In *Proceedings of the 25th International Conference Companion on World Wide Web (WWW '16 Companion)*. ACM, 1033–1034.

[19] Andrei Vlad Sambra, Essam Mansour, Sandro Hawke, Maged Zereba, Nicola Greco, Abdurrahman Ghanem, Dmitri Zagidulin, Ashraf Aboulnaga, and Tim Berners-Lee. 2016. *Solid: A Platform for Decentralized Social Applications Based on Linked Data.* Technical Report. MIT CSAIL and Qatar Computing Research Institute. http://emansour.com/research/lusail/solid_protocols.pdf

[20] Robert Schulze, Tom Schreiber, Ilya Yatsishin, Ryadh Dahimene, and Alexey Milovidov. 2024. ClickHouse: Lightning Fast Analytics for Everyone. *Proceedings of the VLDB Endowment* 17, 12 (2024), 3731–3744. https://doi.org/10.14778/3685800.3685802

[21] Craig D. Weissman and Steve Bobrowski. 2009. The Design of the Force.com Multitenant Internet Application Development Platform. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data (SIGMOD '09)*. ACM, 889–896. https://doi.org/10.1145/1559845.1559942